# LAB 19 — ReactJS Login & Property Drilling

### Professor's Explainer & Student Activity Sheet

**Assigned problem:** Create a ReactJS application with login functionality. Maintain the current user in React state and make that user information available across the application using **property drilling**.

| | |
|---|---|
| **Concepts** | **useState • conditional rendering • props • callback props • property drilling • Bootstrap** |
| **Starting point** | App.jsx renders the Login component. The supplied project contains Login.jsx, LogInHandle.jsx and Dashboard.jsx. |
| **Learning goal** | Understand the movement of data and functions between components rather than memorizing or copying code. |

### Instructor note

**This handout intentionally does not reproduce the program as a copy-paste solution. Students should use the supplied files, trace the data flow, and then implement the extension task.**

## What you should be able to explain after this lab

- Why the current user state belongs in Login.jsx in this version of the application.
- How LogInHandle sends successful login information upward through a callback prop.
- How Login passes the current user downward to Dashboard through a prop.
- Why this is called property drilling: data/functions travel through component props.
- How conditional rendering switches between the login screen and dashboard.
- How Bootstrap components can turn a plain dashboard into a realistic UI.

# 1. Problem Statement

You are given a small React application. The root component renders a Login component. Your job is to make the application behave like a basic authenticated flow: the student enters credentials, the app stores the current user, the dashboard displays that user, and logout clears the user.

**Important: the credentials in this lab are deliberately hard-coded for learning. This is not real authentication. A production application would use a server/API, secure credential handling, sessions or tokens, and protected routes.**

# 2. Understand the Three Main Files

| File | Responsibility | Think of it as… |
|---|---|---|
| Login.jsx | Owns the shared **user state**. Decides whether Login form or Dashboard should be shown. | The controller of the login flow |
| LogInHandle.jsx | Collects name/password and checks the hard-coded credentials. On success it calls the callback received from Login. | The login form |
| Dashboard.jsx | Receives the current user and logout function as props. Displays the user and provides a logout action. | The logged-in screen |

# 3. First Screenshot — Login Screen

The supplied UI is intentionally simple. The important part is not the styling; it is what happens when the user submits the form.

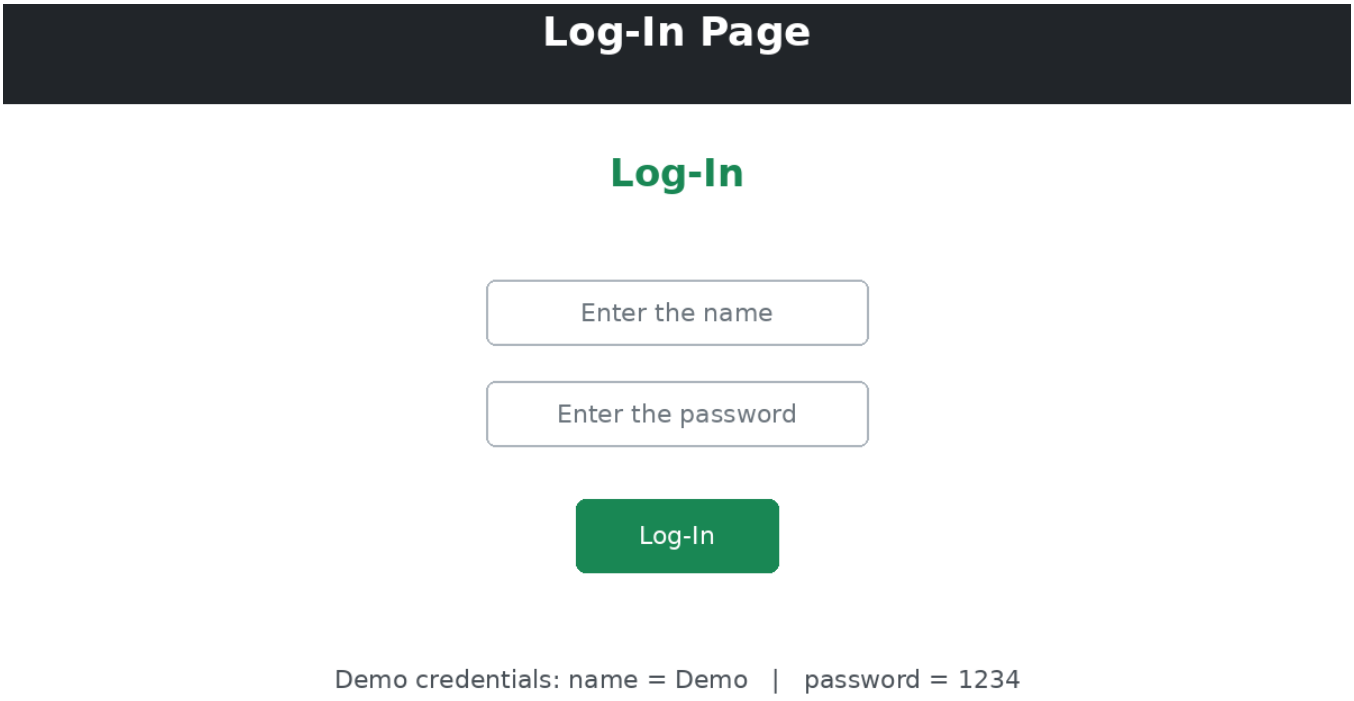

Figure 1. Expected learning view of the login screen. The supplied program checks for the demonstration credentials **Demo / 1234**.

# 4. The Complete Flow

Think of the application as a loop: **input** → **validation** → **state update** → **conditional rendering** → **logout** → **state reset**.

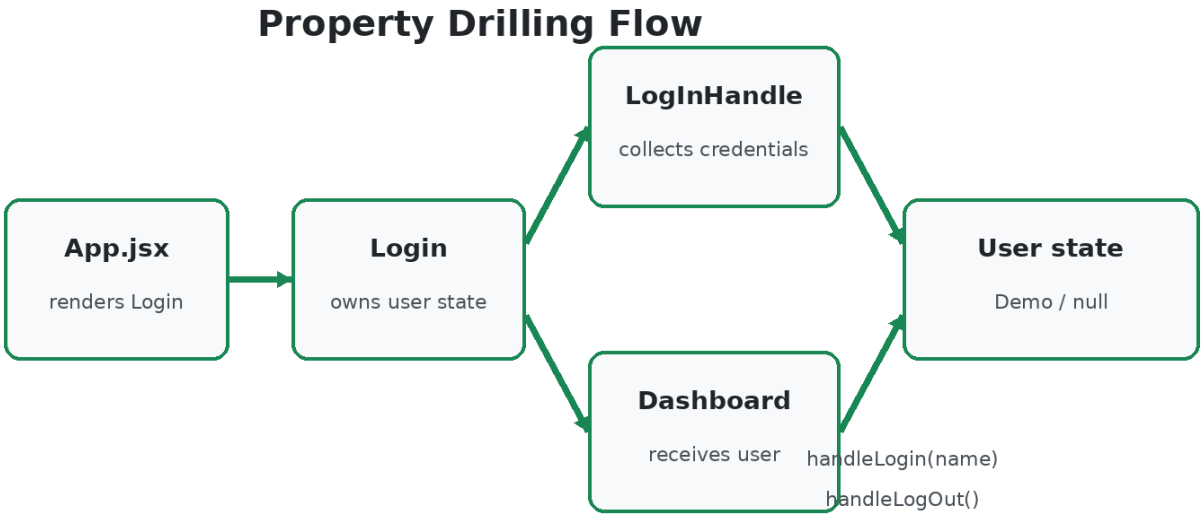


Figure 2. Data and callback flow between the components.

## Step-by-step

• **1 — App.jsx:** renders the Login component. App itself does not need to know the username in this implementation.

• **2 — Login.jsx:** creates a user state variable. At the beginning it is empty/null, meaning nobody is logged in.

• **3 — Login.jsx:** creates a login callback. It accepts the successful login data and updates the user state.

• **4 — Login.jsx:** passes that callback to LogInHandle as a prop. This is the first important property-drilling direction: a function moves downward.

• **5 — LogInHandle.jsx:** keeps temporary form values for name and password. When submitted, it compares them with the lab credentials.

• **6 — Successful login:** LogInHandle calls the callback it received. The callback executes inside Login, so Login updates its own user state.

• **7 — Conditional rendering:** because user is now non-null, Login stops showing the login form and renders Dashboard instead.

• **8 — Dashboard:** receives user as a prop and can display the current user. It also receives the logout callback.

• **9 — Logout:** Dashboard calls the logout callback. Login responds by setting user back to null, which causes the login screen to appear again.

# 5. What Is Property Drilling?

Property drilling means passing data or functions through component props so a component farther down the tree can use them. It is common in small React applications and is useful for learning how data flows.

### In this lab, the direction looks like this:

| Direction | What travels? | Why? |
|---|---|---|
| Login → LogInHandle | login callback | The child needs a way to tell the parent that login succeeded. |
| Login → Dashboard | current user | The dashboard needs to display information about the logged-in user. |
| Login → Dashboard | logout callback | The dashboard needs a way to request that the parent clears the user state. |

### A useful mental model

**The parent component owns the important state. Children do not directly change that state. Instead, the parent gives children functions that they can call. This keeps the source of truth in one place.**

# 6. Why Conditional Rendering Works Here

Login uses the value of user as the switch for the entire screen. When the state is null/empty, the login component is rendered. After a successful login, user contains the current user, so the dashboard is rendered. When logout sets the state back to null, React re-renders and the login screen returns.

### State transition

| Moment | user state | Visible screen |
|---|---|---|
| Application starts | null | Login |
| Valid login | Demo | Dashboard |
| Logout | null | Login |

# 7. Dashboard After Login

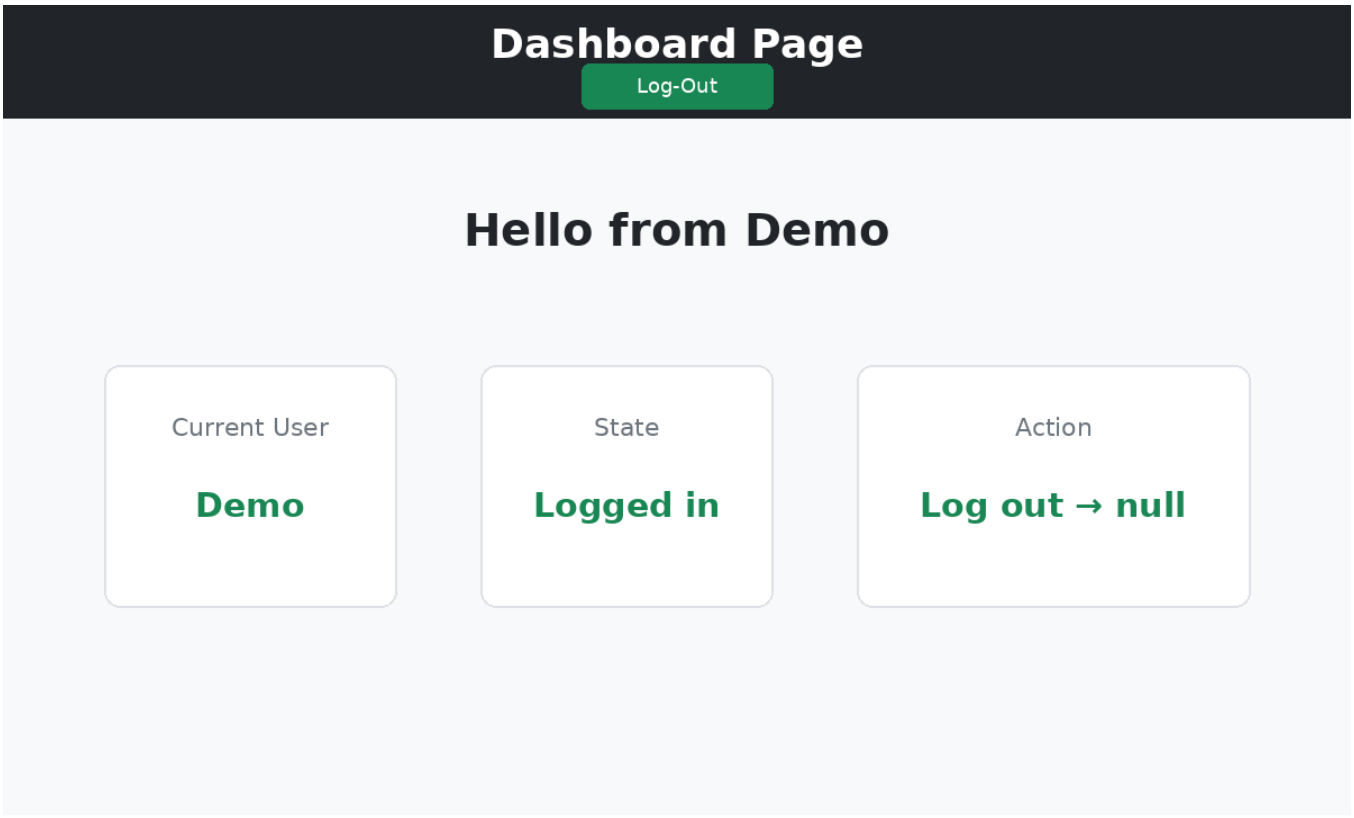


Figure 3. Expected dashboard behavior: the user value reaches Dashboard through props, and the logout action travels back through a callback.

# 8. Trace the Program Like a Debugger

When students are confused, do not start by rewriting the code. Trace one event at a time:

- Type a name. Ask: which state variable changes?
- Type a password. Ask: where is that value stored?
- Press Log-In. Ask: which function runs first?
- Use the valid credentials. Ask: which callback is invoked?
- Ask: which component owns the user state?
- Ask: which prop carries the user into Dashboard?
- Press Log-Out. Ask: which function is called and which state value changes?
- Watch the UI. Ask: why does React show the login screen again?

## Instructor checkpoint questions

- If Dashboard changed the parent’s user variable directly, what React rule would that violate?
- Why is a callback prop useful when a child needs to cause a state change owned by its parent?
- What would happen if the parent passed the user value but forgot to pass the logout callback?
- Which component should be responsible for deciding whether the user is logged in?

# 9. EXTRA TASK — Build a Bootstrap Dashboard

**Objective:** extend the existing Dashboard so that students learn to use an online component library rather than manually designing every UI element.

**Use Bootstrap components such as a navbar, cards, buttons, badges, alerts, and a responsive grid. Keep the current user flow working.**

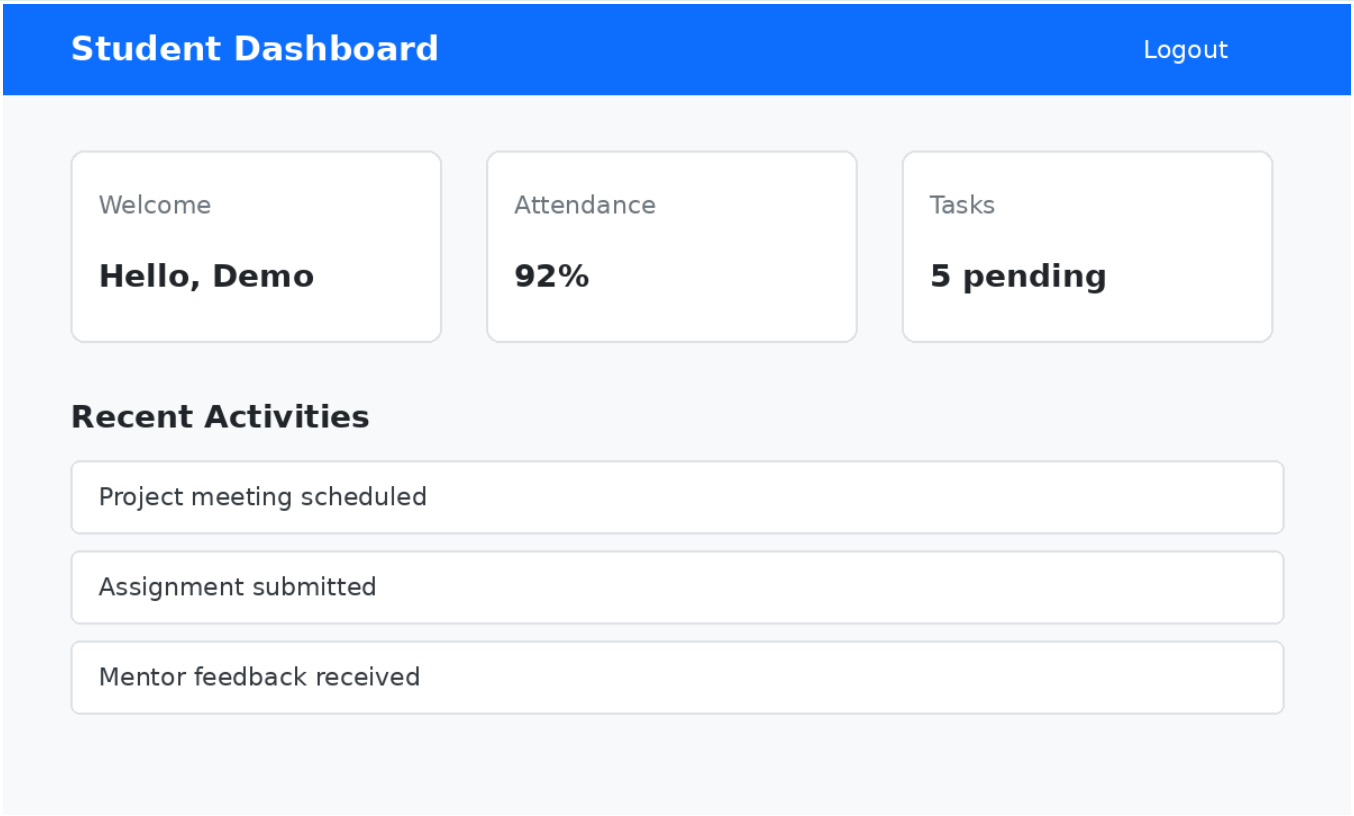


Figure 4. Reference appearance for the extension task. This is a design guide, not a copy-paste implementation.

## Your task requirements

- **1.** Add a Bootstrap navbar/header to the Dashboard.
- **2.** Display the current user's name inside the dashboard using the user prop.
- **3.** Create at least three Bootstrap cards, for example: Attendance, Projects, and Pending Tasks.
- **4.** Add a Bootstrap success/primary button for an action such as “View Profile”.
- **5.** Add a badge or alert showing a small status such as “Logged in”.
- **6.** Make the cards responsive using Bootstrap’s grid classes.
- **7.** Keep the existing Logout functionality. Do not move the user state into Dashboard.
- **8.** Test both flows: valid login → dashboard → logout → login.

# 10. Making an Online Component Library Work

The point of this task is to understand that a UI library is still just HTML + CSS + JavaScript behavior exposed through reusable classes/components. Students should learn to read documentation, identify the component they need, and adapt it to their own data.

## Suggested documentation workflow

- Open the official Bootstrap documentation.
- Find the component you need instead of searching for a complete dashboard to copy.
- Read the example and identify which classes create the layout, spacing, colors, and responsive behavior.
- Add the classes to your own JSX structure.
- Replace static text with your React data, especially the current user.
- Test the page at different browser widths.
- If a component requires JavaScript behavior, check the Bootstrap documentation for the required setup.

**Student challenge: do not copy an entire dashboard template. Build the dashboard from individual Bootstrap components.**

# 11. Common Mistakes to Check

**Login screen never changes:** Check whether the successful-login callback actually updates the user state.

**Dashboard shows undefined user:** Check that the parent passes the user prop and that Dashboard reads the same prop name.

**Logout button does nothing:** Check that Dashboard receives the logout callback and actually invokes it when clicked.

**Page refresh loses the user:** Expected in this simple lab: state is in memory only. Persistence is a separate topic.

**Password is visible:** Improve the password field by using the appropriate password input type.

**Form refreshes the page:** When using a form submit handler, prevent the browser's default submission behavior.

**Login accepts unexpected credentials:** Verify the comparison logic and test invalid input deliberately.

**Bootstrap styles do not appear:** Check that Bootstrap has been installed/imported or included according to the setup you are using.

# 12. Submission / Assessment Checklist

■ Login component owns the current user state.

■ Successful login changes the user state.

■ Invalid credentials do not enter the dashboard.

■ Dashboard receives the user through props.

■ Dashboard displays the current user.

■ Logout clears the user state.

■ Property drilling can be explained verbally with the component flow.

■ Dashboard uses multiple Bootstrap components.

■ Dashboard layout is responsive.

■ Student can explain the program without reading a copied solution.

# 13. Short Viva Questions

• What is the difference between state and props?

• Why is user state kept in Login rather than directly inside Dashboard?

• What is property drilling?

• Why do we pass functions as props?

• What causes React to render Dashboard instead of LogInHandle?

• What happens to the UI when user becomes null?

• How would you share the current user if the application became much larger?

## Extension discussion

If the component tree becomes deep and many unrelated components need the same user information, property drilling can become cumbersome. At that point, students can explore React Context or a state-management library. For this lab, however, **property drilling is the required learning objective**.

**Design principle reminder: use the simplest solution that satisfies the current requirement. Do not introduce global state management just because it exists.**

# Lab outcome

By completing this exercise, students should be able to look at a React component tree and answer three questions: **Who owns the state? Who needs the data? Which prop or callback moves the information between them?** Once those questions are clear, the code becomes much easier to understand.